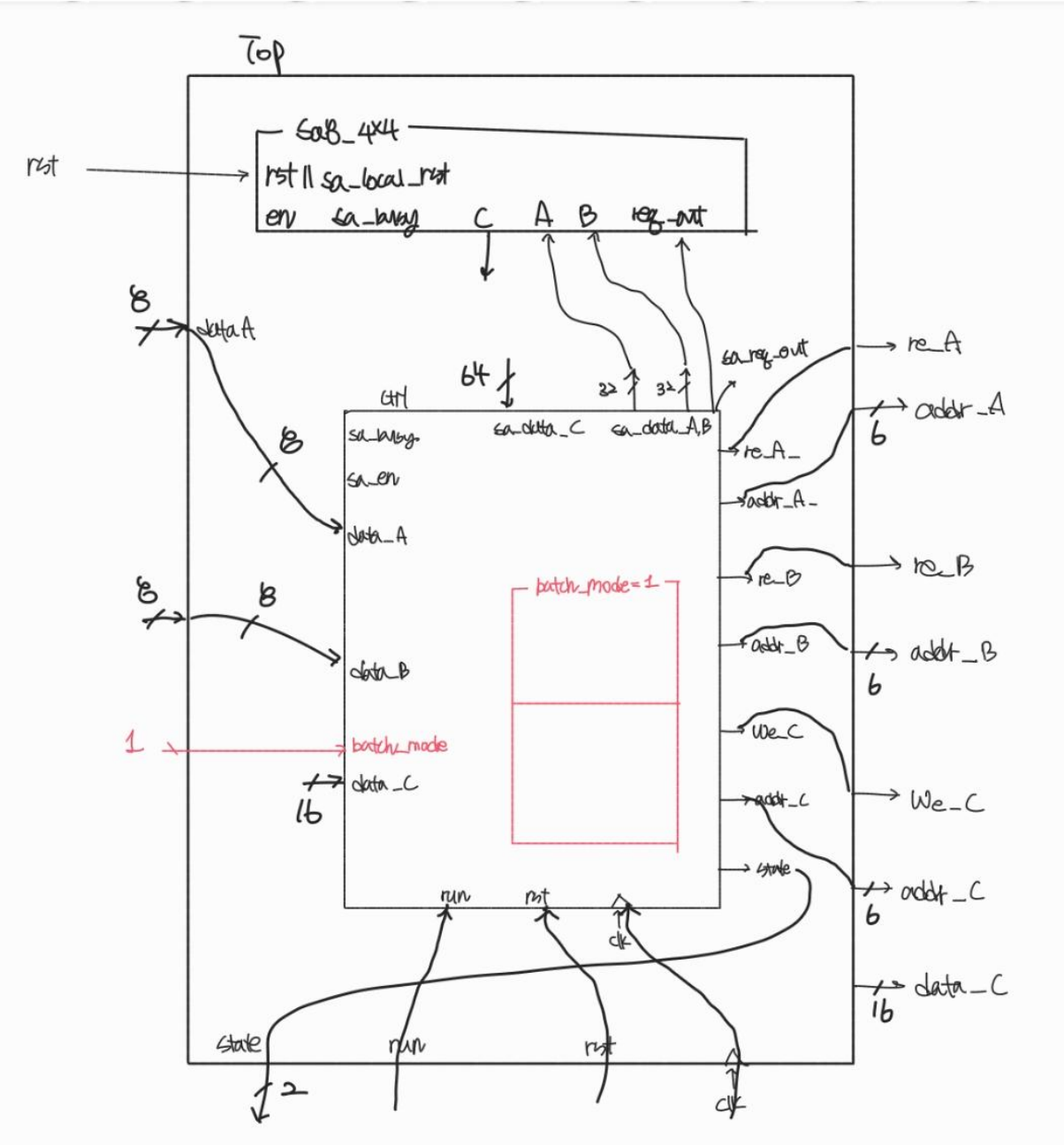
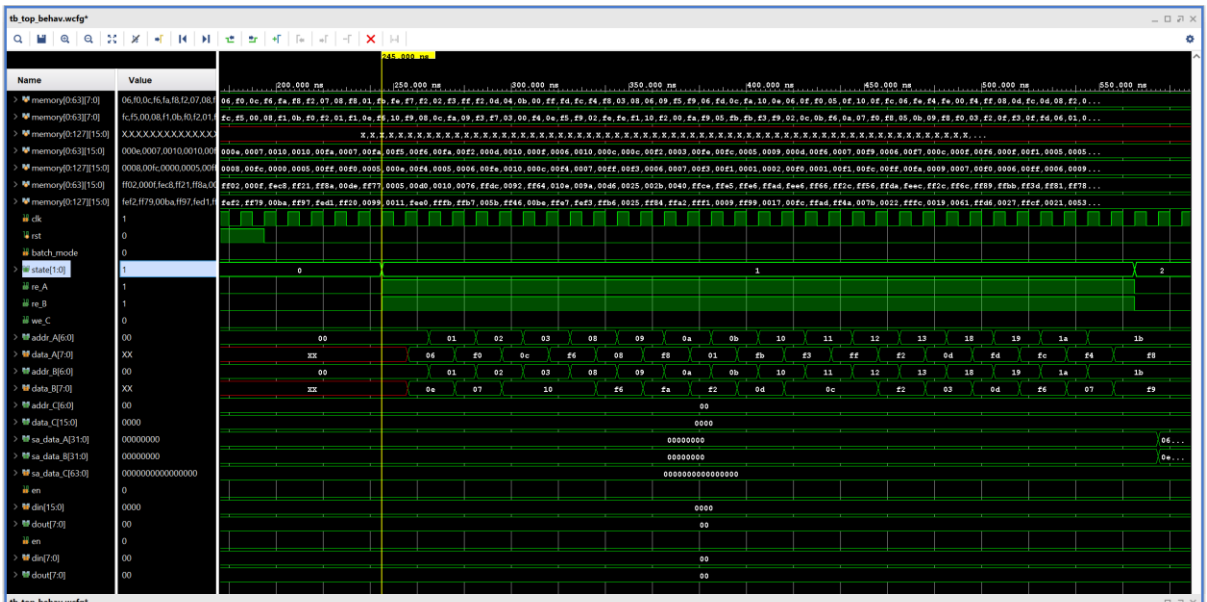
**Final Project Report**

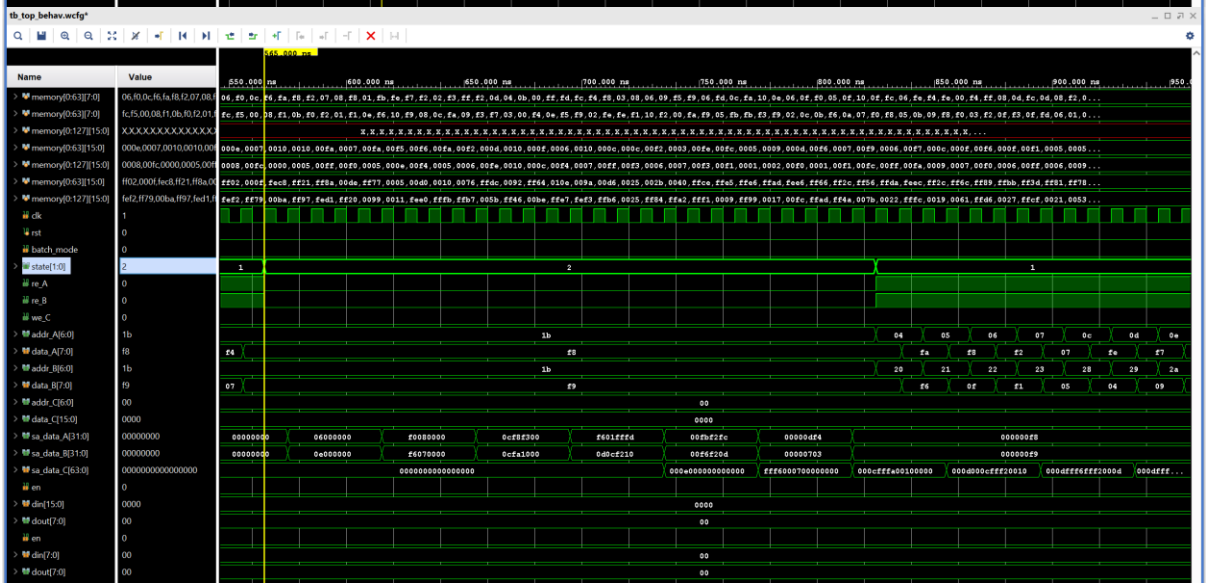
**202111052 한민준**

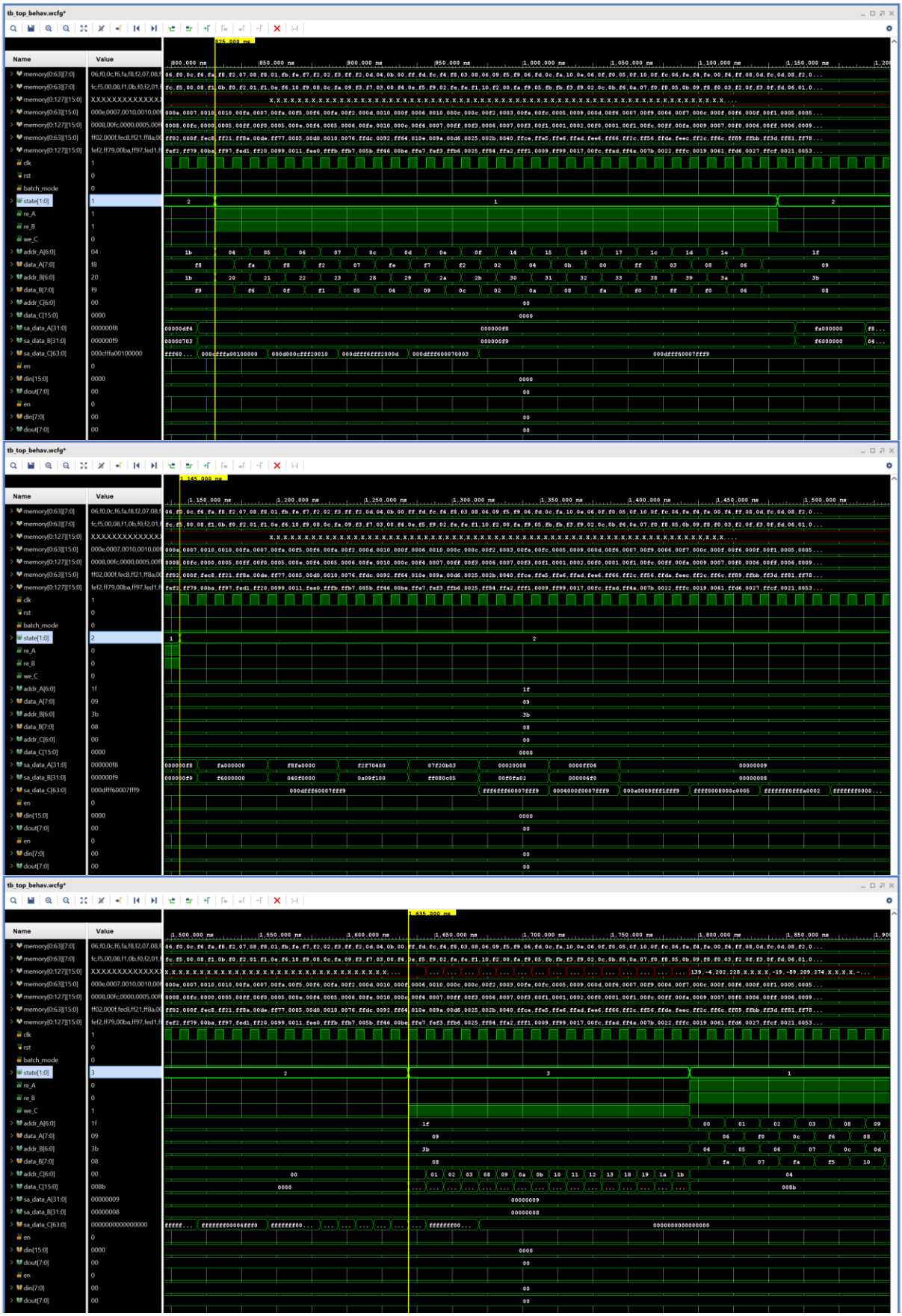


## -Base Task (Batch 8)

## 1) X * W → FC1 → X1



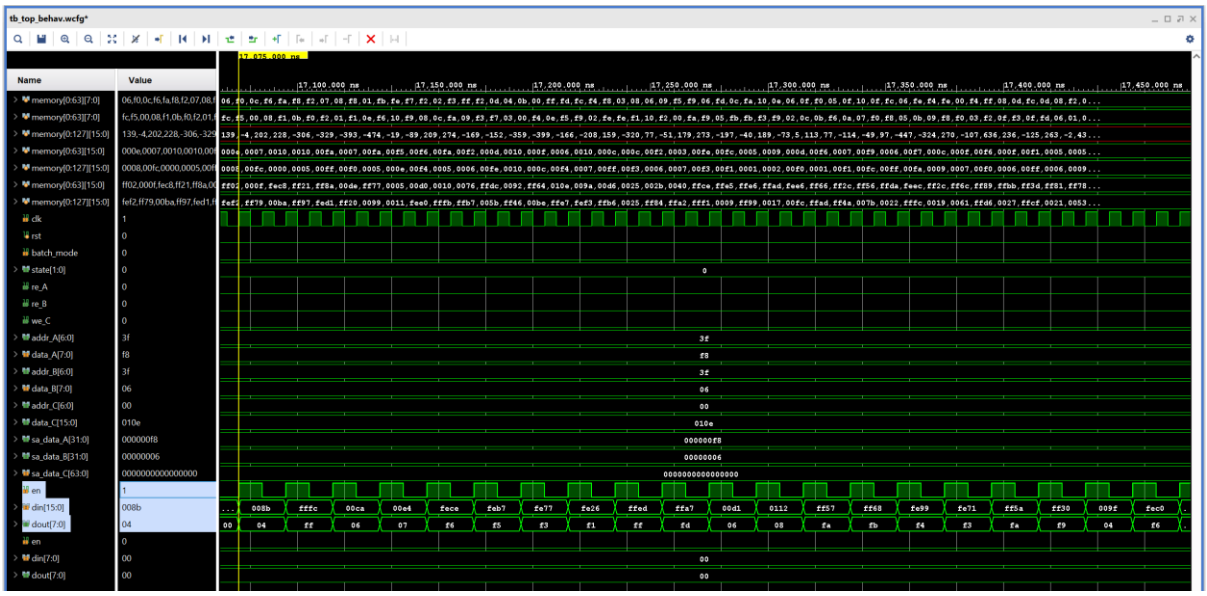



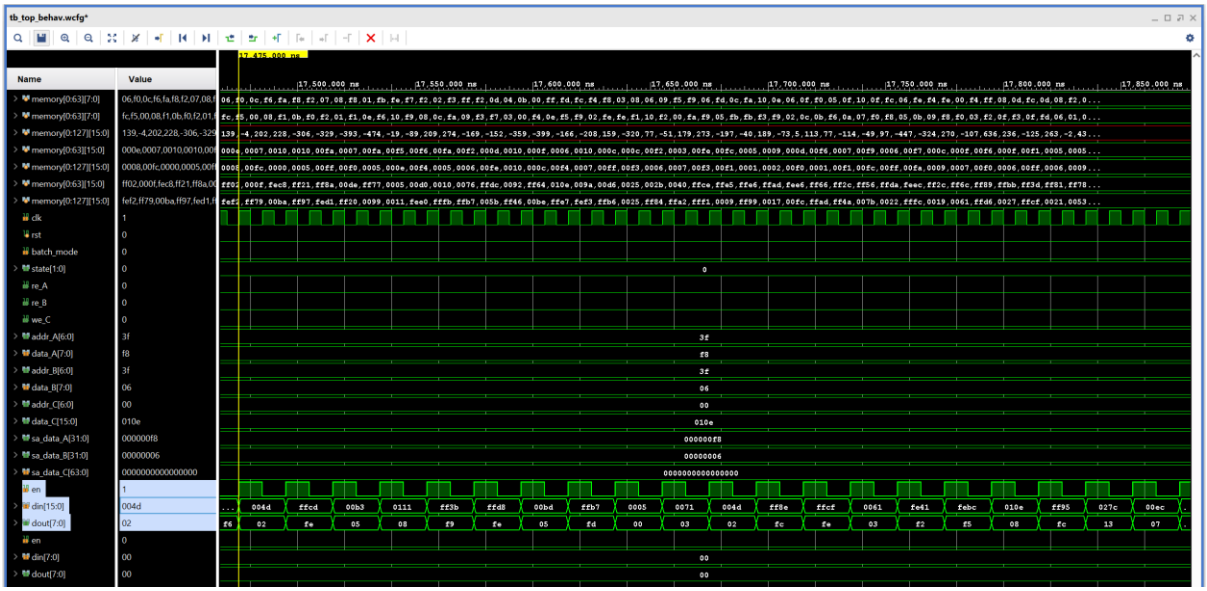

tb_top_behav.wcfg*
Name
Value
memory[0:63][7:0]
memory[0:63][7:0]
memory[0:127][15:0]
memory[0:63][15:0]
memory[0:127][15:0]
memory[0:63][15:0]
memory[0:127][15:0]
clk
rst
batch_mode
state[1:0]
re_A
re_B
we_C
addr_A[6:0]
data_A[7:0]
addr_B[6:0]
data_B[7:0]
addr_C[6:0]
data_C[15:0]
sa_data_A[31:0]
sa_data_B[31:0]
sa_data_C[63:0]
en
din[15:0]
dout[7:0]
en
din[7:0]
dout[7:0]

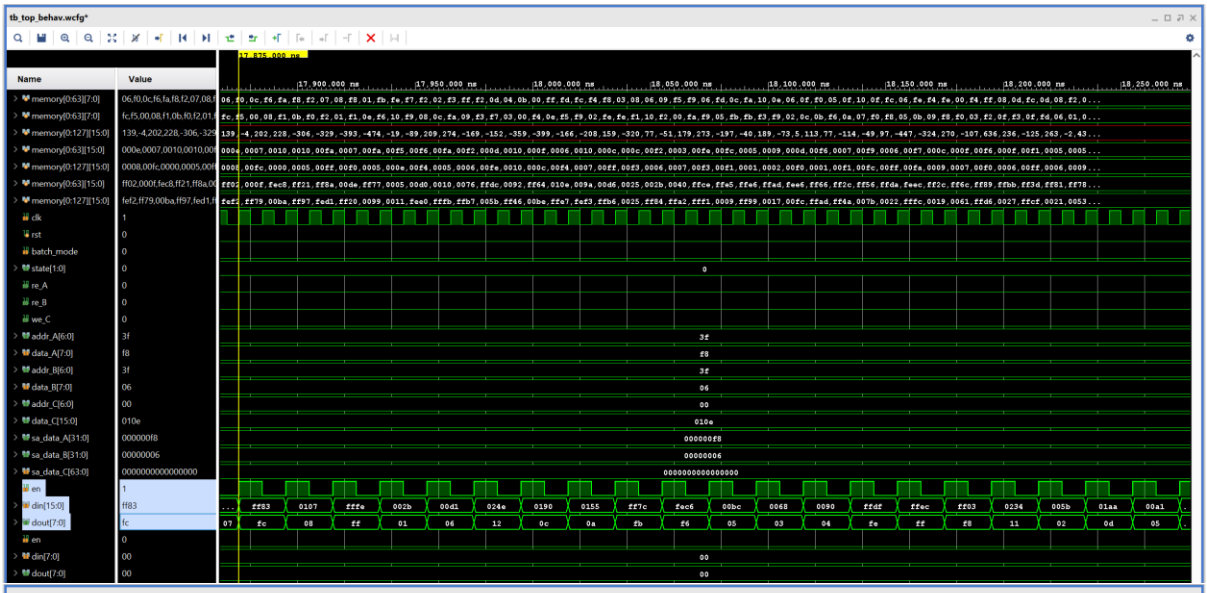
위 파형은 Batch = 8 조건에서 FC1 연산, 즉 W(8x8)와 X(8x8)의 행렬 곱셈이 수행된 과정을 보여준다. 이때 연산은 이전 Assignment 4에서와 같이 총 4개의 타일로 분할하여 처리하였으며, 각 타일은 순차적으로 Tile0 → Tile1 → Tile2 → Tile3의 순서로 계산된다. Batch가 8일 때에는 기존 구조를 그대로 사용할 수 있으므로, 별도의 구조 변경 없이 동일한 연산 흐름을 유지하였다.

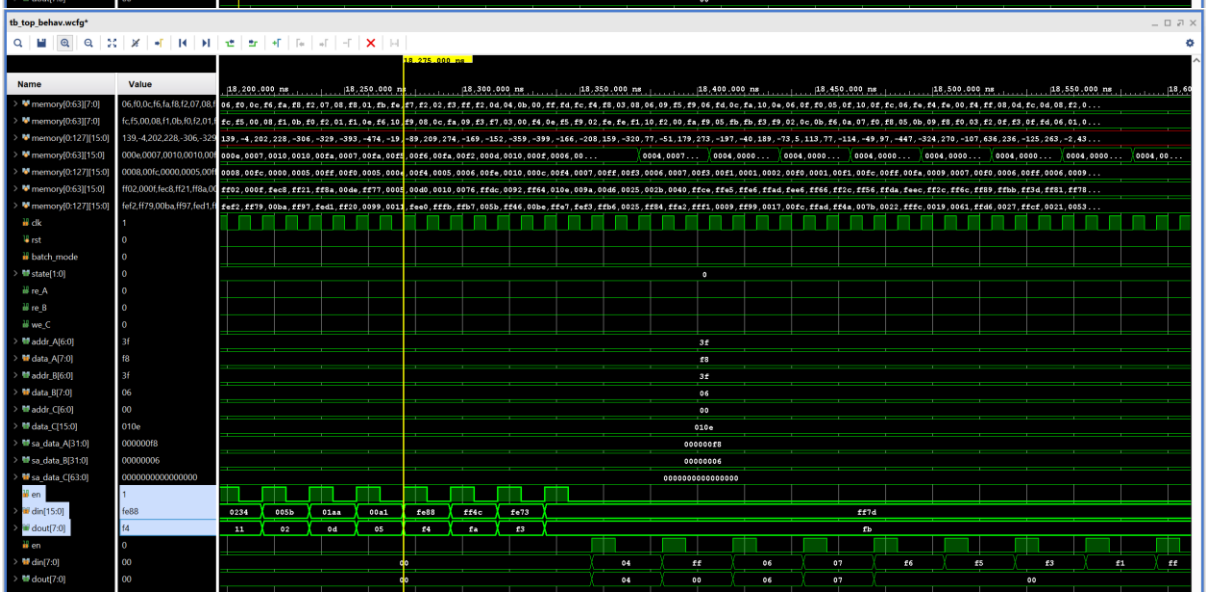
하나의 타일을 처리하는 동안에는 LOAD → COMPUTE → LOAD → COMPUTE → WRITE의 단계가 반복되며, 각 타일 연산이 완료될 때마다 결과 데이터를 메모리에 기록한다. 현재 파형에는 Tile0의 연산 흐름만 제시되어 있으나, 나머지 3개의 타일도 같은 방식으로 진행된다. 단, 이때는 Load와 Write 단계에서 사용하는 메모리 주소만 달라지도록 구성하여, 전체적으로 8×8 크기의 출력 행렬 X1이 완성된다. 이 결과는 데이터 하나당 16비트로 표현되며, FC1 레이어의 출력으로 사용된다.

**2) X1 → NORM → X2**




tb_top_behav.wcfg*
Name
Value
memory[0:63][7:0]
memory[0:63][7:0]
memory[0:127][15:0]
memory[0:63][15:0]
memory[0:127][15:0]
memory[0:63][15:0]
memory[0:127][15:0]
clk
rst
batch_mode
state[1:0]
re_A
re_B
we_C
addr_A[6:0]
data_A[7:0]
addr_B[6:0]
data_B[7:0]
addr_C[6:0]
data_C[15:0]
sa_data_A[31:0]
sa_data_B[31:0]
sa_data_C[63:0]
en
din[15:0]
dout[7:0]
en
din[7:0]
dout[7:0]

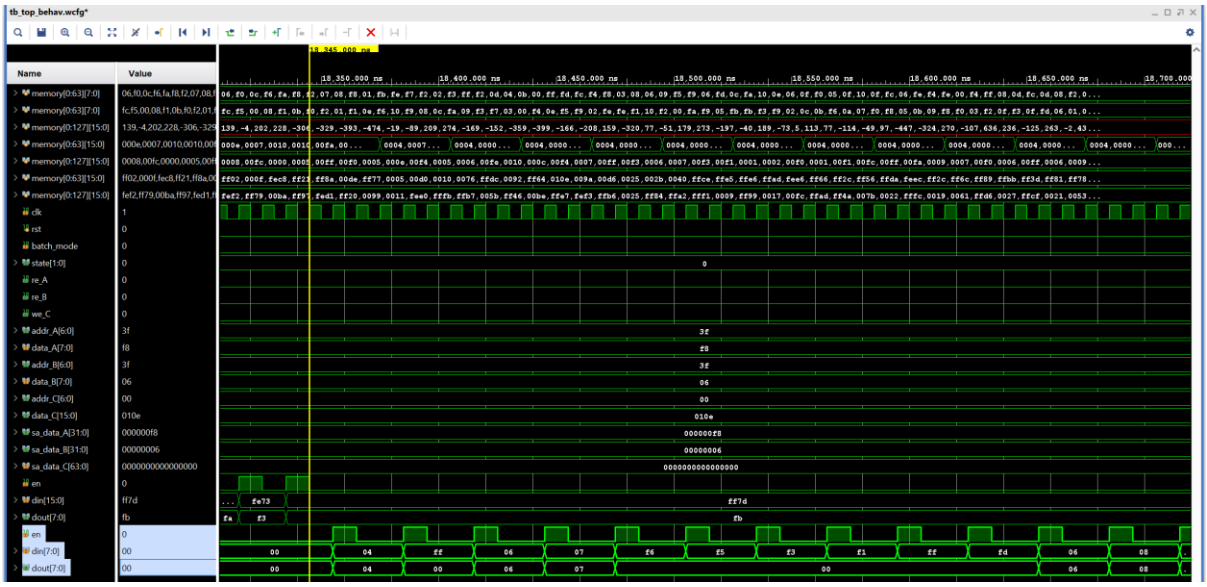

tb_top_behav.wcfg*
Name
Value
memory[0:63][7:0]
memory[0:63][7:0]
memory[0:127][15:0]
memory[0:63][15:0]
memory[0:127][15:0]
memory[0:63][15:0]
memory[0:127][15:0]
clk
rst
batch_mode
state[1:0]
re_A
re_B
we_C
addr_A[6:0]
data_A[7:0]
addr_B[6:0]
data_B[7:0]
addr_C[6:0]
data_C[15:0]
sa_data_A[31:0]
sa_data_B[31:0]
sa_data_C[63:0]
en
din[15:0]
dout[7:0]
en
din[7:0]
dout[7:0]

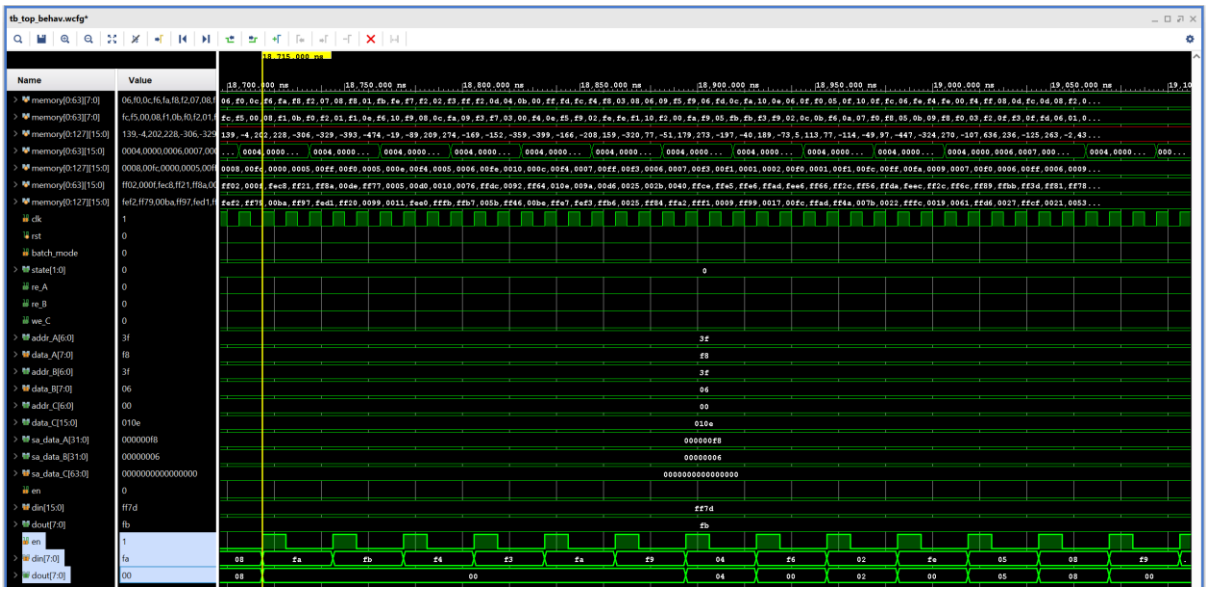

tb_top_behav.wcfg*
Name
Value
memory[0:63][7:0]
memory[0:63][7:0]
memory[0:127][15:0]
memory[0:63][15:0]
memory[0:127][15:0]
memory[0:63][15:0]
memory[0:127][15:0]
clk
rst
batch_mode
state[1:0]
re_A
re_B
we_C
addr_A[6:0]
data_A[7:0]
addr_B[6:0]
data_B[7:0]
addr_C[6:0]
data_C[15:0]
sa_data_A[31:0]
sa_data_B[31:0]
sa_data_C[63:0]
en
din[15:0]
dout[7:0]
en
din[7:0]
dout[7:0]

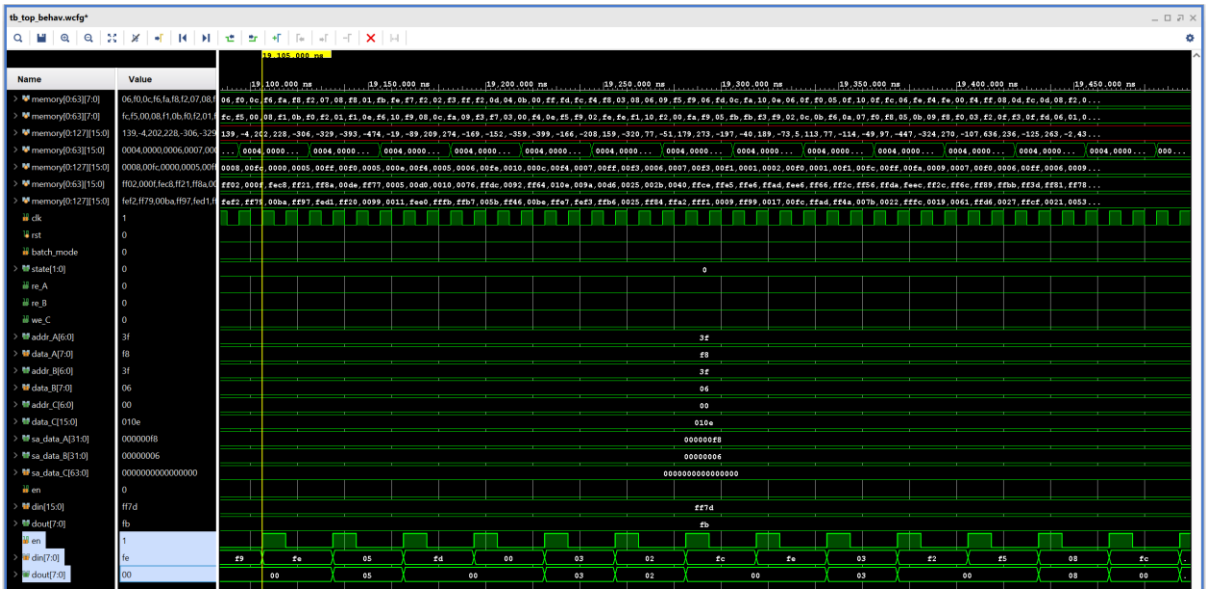
해당 파형은 FC1 연산 결과인 X1을 정규화(normalization)하는 과정을 보여준다. 강조된 세 개의 신호는 norm 모듈 내부에서 활용되는 주요 제어 신호이며, X1은 16비트 크기의 값으로 구성되어 있어 후속 연산에서 연산량이 과도하게 증가하는 문제를 유발할 수 있다. 이를 방지하기 위해 정규화 작업을 수행하게 되며, 이는 곧 X1을 5비트 오른쪽으로 시프트(즉, 32로 나눈 것)하여 X2로 사용하는 방식이다.

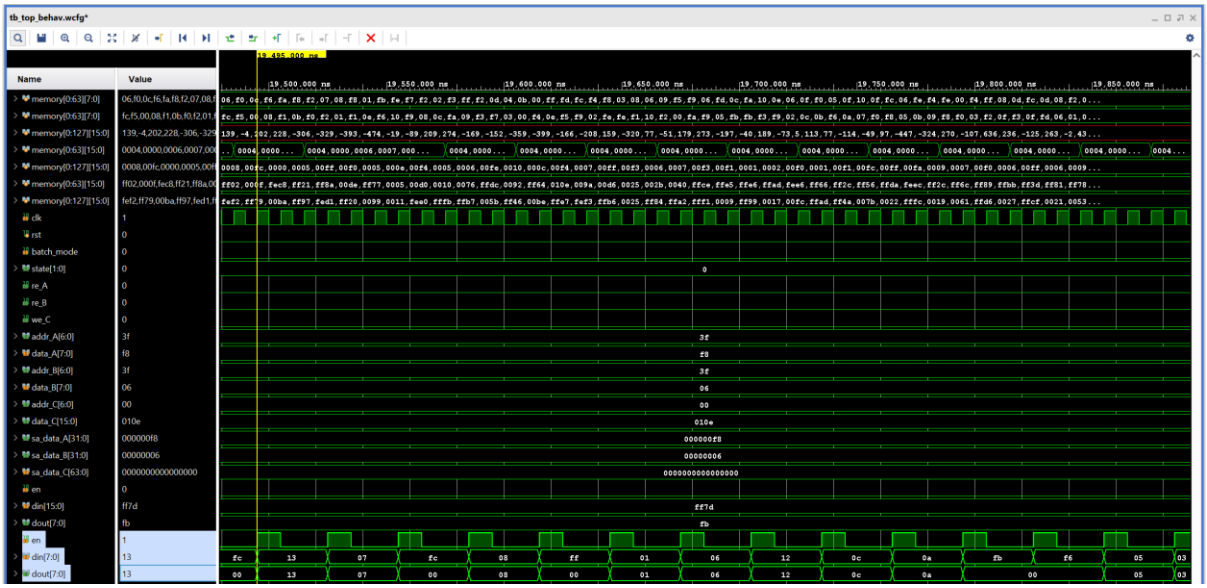
구현 측면에서는 X1의 입력(din) 중 상위 비트인 [12:5] 구간의 8비트만을 추출하여 출력(dout)으로 전달하도록 하였고, 이 출력값들이 새로운 정규화된 벡터 X2를 형성하게 된다. 파형을 통해 확인한 결과, 16비트였던 입력값들이 norm 과정을 거치면서 8비트로 줄어든 것을 dout에서 명확히 확인할 수 있었다.

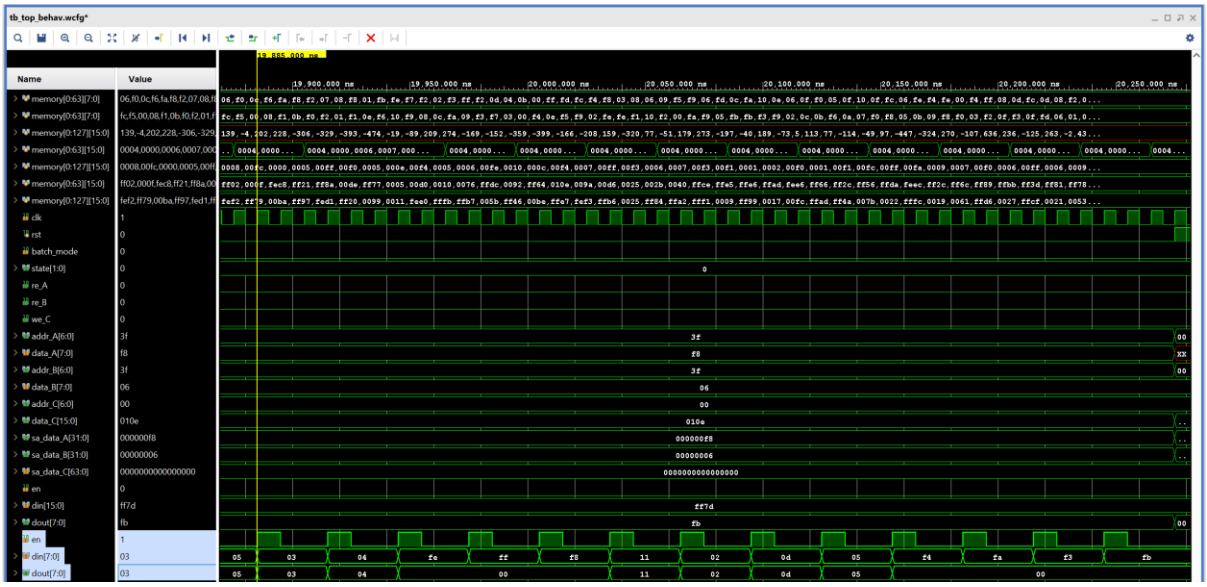
### 3) X2 → ReLU → X3



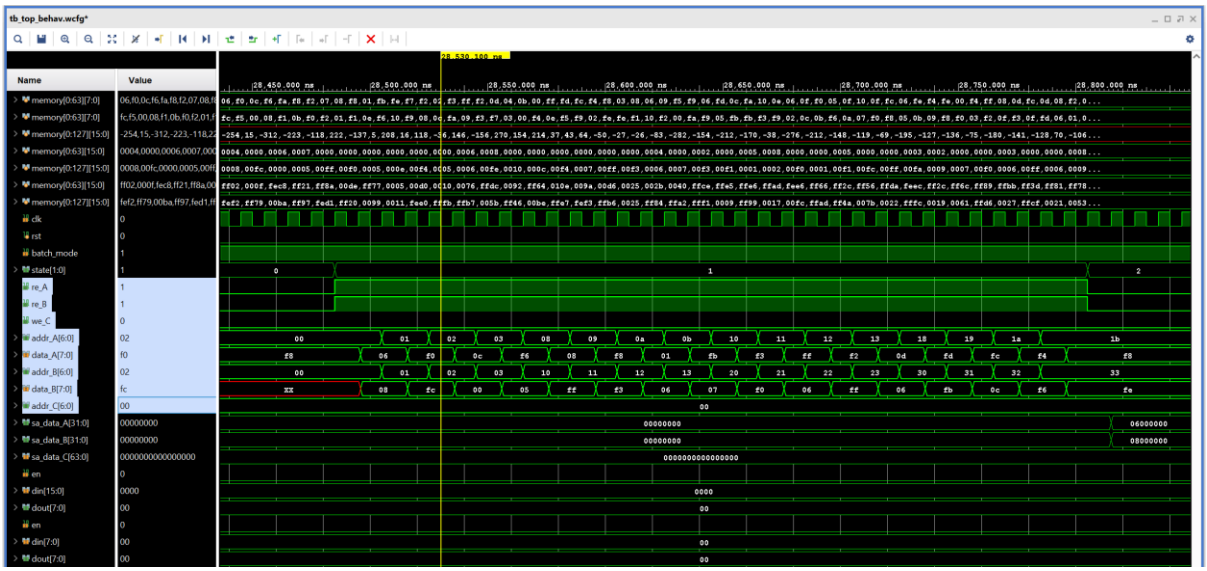

tb_top_behav.wcfg*
Name
Value
clk
rst
batch_mode
state[1:0]
re_A
re_B
we_C
addr_A[6:0]
data_A[7:0]
addr_B[6:0]
data_B[7:0]
addr_C[6:0]
data_C[15:0]
sa_data_A[31:0]
sa_data_B[31:0]
sa_data_C[63:0]
en
din[15:0]
dout[7:0]
en
din[7:0]
dout[7:0]


tb_top_behav.wcfg*
Name
Value
clk
rst
batch_mode
state[1:0]
re_A
re_B
we_C
addr_A[6:0]
data_A[7:0]
addr_B[6:0]
data_B[7:0]
addr_C[6:0]
data_C[15:0]
sa_data_A[31:0]
sa_data_B[31:0]
sa_data_C[63:0]
en
din[15:0]
dout[7:0]
en
din[7:0]
dout[7:0]


tb_top_behav.wcfg*
Name
Value
clk
rst
batch_mode
state[1:0]
re_A
re_B
we_C
addr_A[6:0]
data_A[7:0]
addr_B[6:0]
data_B[7:0]
addr_C[6:0]
data_C[15:0]
sa_data_A[31:0]
sa_data_B[31:0]
sa_data_C[63:0]
en
din[15:0]
dout[7:0]
en
din[7:0]
dout[7:0]

이 파형은 정규화된 X2 데이터를 ReLU 모듈을 통해 처리하는 과정을 나타낸다. ReLU 연산에서는 입력 값의 부호 비트(MSB)를 기준으로 동작하며, MSB가 1인 경우에는 해당 값을 0으로, MSB가 0인 경우에는 값을 그대로 유지하도록 설계하였다. 모듈이 활성화되는 조건은 en 신호가 high일 때이며, 이때 ReLU 기능이 실제로 적용되어 결과 값이 출력된다. 이러한 방식으로 음수는 제거되고, 양수 값만이 다음 연산에 전달될 수 있도록 구성되었다.

**4) W2 * X3 → FC2 → Y**



해당 파형은 ReLU 처리를 마친 결과값 X3를 W2와 곱하여 FC2 연산을 시작하는 시점을 보여준다. 현재 Batch 크기가 8로 설정되어 있기 때문에, FC1 때와 마찬가지로 이전 과제에서 사용했던 SA 모듈을 그대로 활용하여 FC2 연산도 동일한 방식으로 수행하게 된다.

FC2 연산이 동일한 방식으로 모두 완료되면, 최종 출력값인 Y가 생성된다. 이후 이 결과는 testbench에 정의된 task를 통해 검증되며, inout_base//y_hex.txt 파일에서 불러온 정답 데이터와 메모리에 저장된 연산 결과를 비교함으로써 전체 연산 과정(FC1 → Normalization → ReLU → FC2)이 정확하게 수행되었는지를 확인할 수 있도록 구성되어 있다.

```
=== Comparison Complete ===
Total errors: 0 out of 64 elements
*** TEST PASSED - ALL RESULTS MATCH! ***
=== Final Test Result: SUCCESS ===
```

시뮬레이션을 통해 총 64개의 출력 결과가 Reference 값과 정확히 일치하는 것

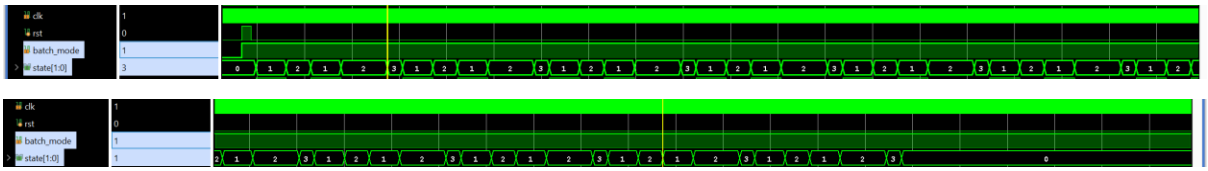
을 확인하였으며, 이를 통해 batch size가 8인 경우의 전체 연산 과정이 정상적으로 수행되었음을 검증할 수 있었다.

**-Extra Task (Batch 16)**

**1) W1 * X → FC1 → X1**

Batch size가 8인 경우의 연산이 모두 끝나면, Batch_mode 신호를 1로 설정하여 Batch size가 16인 연산으로 전환하게 된다. 이때 가장 큰 변화는 입력 X와 출력 Y의 크기가 기존의 8×8에서 8×16으로 확장된다는 점이다. 이에 따라 Tiling 횟수도 4회에서 8회로 증가하게 되며, txt 파일에 저장된 데이터가 row-wise 형태로 정렬되어 있기 때문에, 타일링을 위한 메모리 주소 계산 방식에도 약간의 수정이 필요하다.

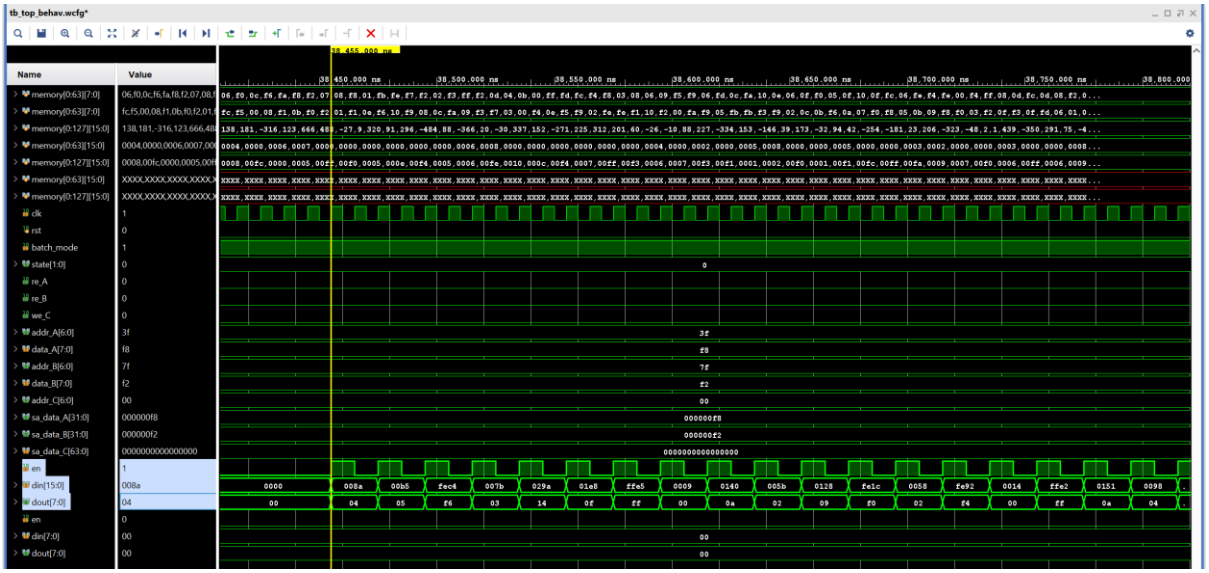
이러한 구조적 변경을 고려하여, 8×16 크기의 행렬을 두 개의 8×8 블록으로 분할한 뒤, 각각을 다시 4개의 타일로 나누어 처리하는 방식으로 연산을 진행하였다.




시뮬레이션 파형을 통해 확인해 보면, state 값이 3일 때마다 하나의 tile 연산이 완료된다는 것을 알 수 있으며, 이를 기준으로 FC1 과정에서 총 8개의 tile 연산이 순차적으로 수행되었음을 확인할 수 있다.

**2) X1 → Norm → X2**

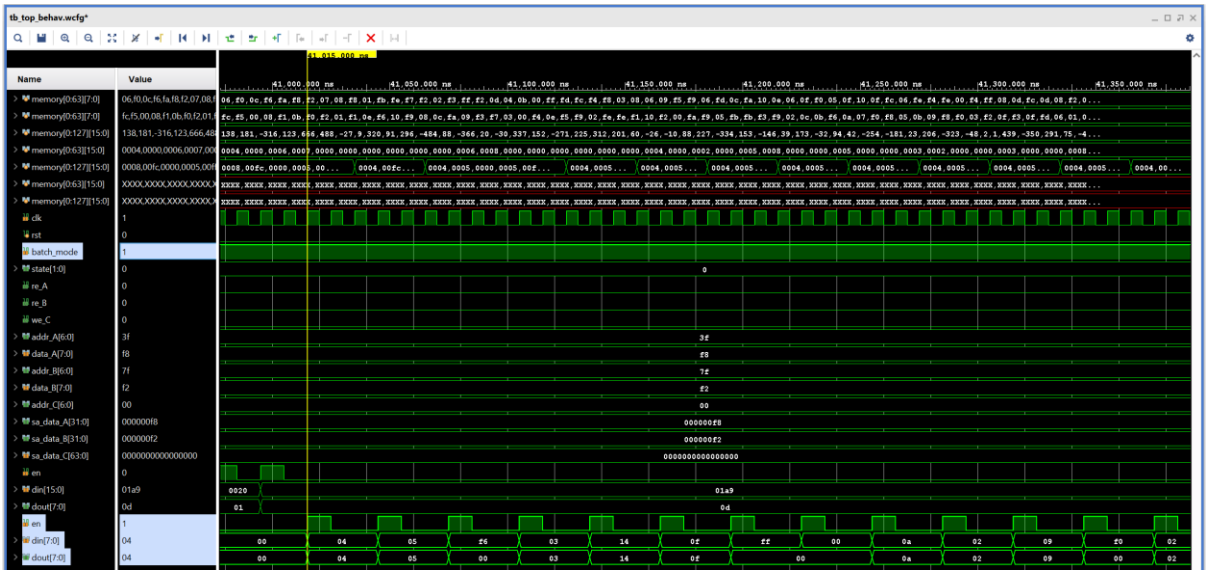
Norm의 경우 Batch 16일때와 8일때가 동일하다. Normalization을 하기 위해 16bit의 data를 5비트만큼 right shift를 하여, 8비트를 취하게 된다. [12:0]의 data 중 [12:5] 8비트를 취하게 됨으로써 Normalization이 이루어진다.



en, din[15:0], dout[7:0] 파형을 통해 din(16bit)이 dout(8bit)로 Norm된 것을 확인 가능하다.

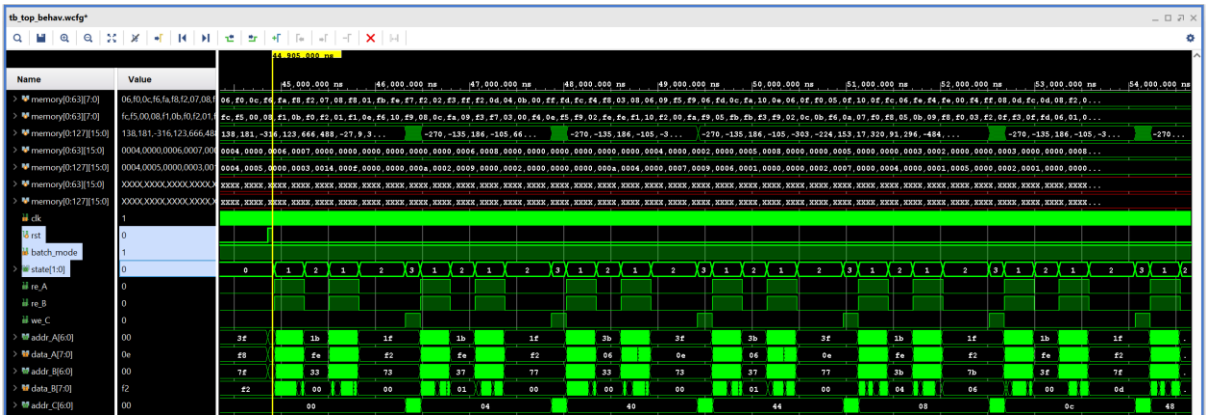
### 3) X2 → ReLU → X3

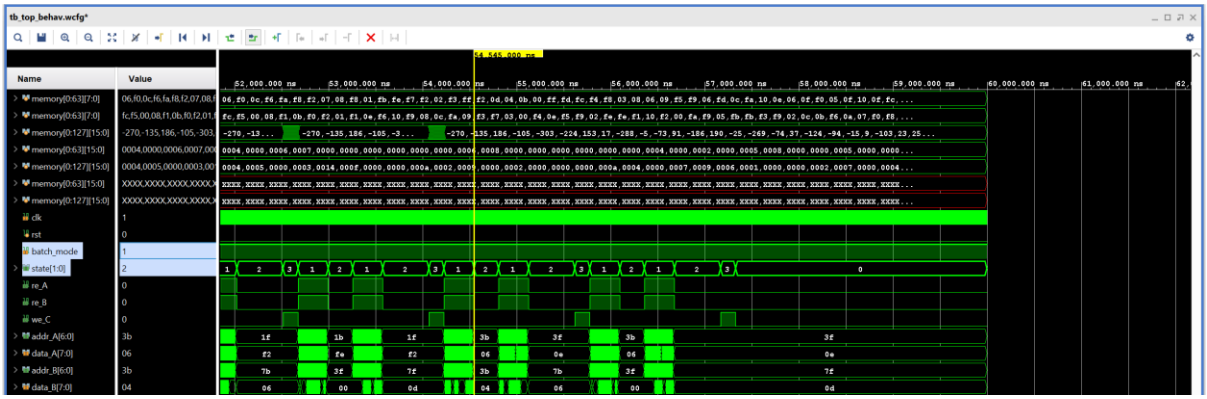
ReLU 또한 8-batch와 16-batch의 진행이 동일하다. MSB가 1이면 dout=0, MSB가 0이면 dout이 din값을 그대로 취한다. 음수를 제거하는 효과를 가져가게 된다.



### 4) W2 * X3 → FC2 → Y

FC2 연산은 앞서 했던 16-batch에 대한 FC1연산과 같이 이루어진다. 8x16 matrix를 만들기 위해 tile을 8개를 만들고, 이에 대해 연산을 진행하게 된다.




위 파형 결과를 통해 LOAD -> COMPUTE -> LOAD -> COMPUTE -> WRITE 가 진행될 때, WRITE는 stage3이고, tile을 8개로 만들어놓았기 때문에, stage3가 총 8번 활성화되게된다.

```
=== Comparison Complete ===
Total errors: 0 out of 128 elements
*** TEST PASSED - ALL RESULTS MATCH! ***
=== Final Test Result: SUCCESS ===
```

최종적으로 Batch size 16에 대한 모든 연산을 마친 후, 생성된 결과값이 reference 데이터가 저장된 메모리의 해당 주소값과 정확히 일치함을 확인함으로써, 전체 연산이 정상적으로 수행되었음을 볼 수 있다.

### 1) What is the peak bandwidth of your design in [MB/s]?

Peak Bandwidth를 계산하기 위해서는, 하나의 클럭 사이클 동안 메모리에서 실제로 전달되는 데이터의 총량을 기준으로 분석할 수 있다. 본 시스템에서는 입력 데이터인 W와 X가 각각 1Byte 크기를 갖고 있으며, 출력 결과인 Y는 2Byte로 구성되어 있다.

메모리 접근은 en 신호와 함께 re(read enable) 또는 we(write enable) 신호의 조합으로 제어된다. en이 1인 상태에서 we가 0이면 re 신호가 활성화되어 데이터를 읽게 되고, 반대로 we가 1이면 쓰기 동작이 수행된다. 이러한 구조로 인해 메모리에서는 읽기와 쓰기를 동시에 처리할 수 없으며, 한 사이클에는 읽기 또는 쓰기 중 한 작업만 수행할 수 있다.

읽기 동작의 경우 W와 X 각각 1Byte씩 총 2Byte를 읽게 되고, 쓰기 시에는 Y의 2Byte가 전송된다. 따라서 읽기일 때의 Bandwidth는 (1B + 1B) / 10ns = 200MB/s, 쓰기일 때도 2B / 10ns = 200MB/s가 되며, 이 시스템의 최대 Bandwidth는 200MB/s로 계산된다.

**2) What is the peak performance of your design in [GOPS]?**

해당 시스템의 SA 모듈은 총 16개의 systolic element로 구성되어 있으며, 각 SE는 하나의 사이클 동안 2번의 OP를 수행하게 된다.(1번의 곱셈, 1번의 덧셈). 전체적으로는 16개의 SE가 동시에 동작하여 총 32OPs/cycle의 연산 용량을 가지게 된다.

Peak Performance = (한 사이클 당 가능한 OP 수) x (clk frequency) 로써 구할 수 있다. 이 시스템의 clk freq가 100MHz(T=10ns)이므로, Peak Performance = 32 OPs/cycle × $100\times10^6$ cycle/s = **3.2 GOPS**가 된다.

**3) Estimate the overall latency of your design in [us].**

**<8-Batch>**

FC1 Latency:

Tile 0 : Stage 1 (320ns)

→ Stage 2 (260ns)

→ Stage 1 (320ns)

→ Stage 2 (490ns)

→ Stage 3 (160ns)

= Total 1550ns for one tile.

So, Overall Latency for FC1 = 1550 x 4 = 6200ns

Normalization Latency : 1280ns

ReLU Latency : 1920ns

FC2 Latency : same as FC1 => 6200ns

**Overall Latency for Batch 8** = 6200 + 1280 + 1920 + 6200 = 15600ns = 15.6us

**<16-Batch>**

FC1 Latency : (1550ns per one tile) * 8tiles = 12400ns

Normalization Latency : 2560ns

ReLU Latency : 3840ns

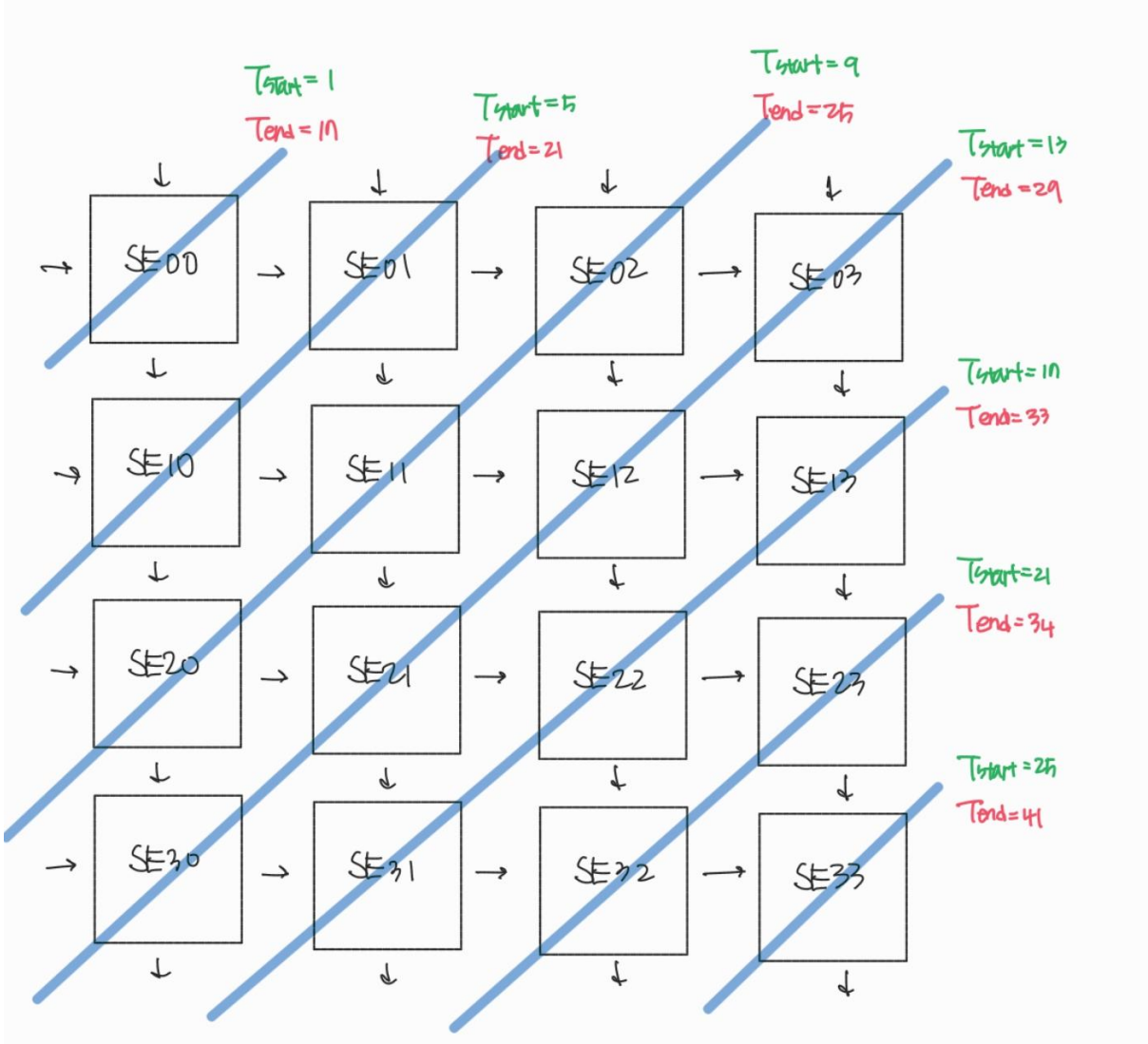
FC2 Latency : same as FC1 => 12400ns

**Overall Latency for Batch 16** = 12400+ 2560 + 3840 + 12400 = 31200ns = 31.2us

**4) Estimate the utilization ratio of your design in [%].**

Utilization Ratio는 시스템이 실제로 수행한 연산량을 이론적으로 가능한 최대 연산량으로 나눈 값으로 정의된다. 본 시스템에서는 하나의 SE가 한 사이클에 곱셈 1회, 덧셈 1회를 수행하므로 총 2 OPs가 발생하며, 총 16개의 SE로 구성된 SA 모듈에서는 이론적으로 사이클당 32 OPs를 처리할 수 있다.

SA 모듈이 최종 출력값을 하나 생성하는 데 필요한 전체 사이클 수는 다음과 같다.




T=0: SE00에 첫 입력값이 들어온다.
T=1: SE00에서 두 input의 하위 4비트끼리 곱셈을 한 결과가 더해지게 된다.
T=2: SE00에서 두 입력값 중 하나의 상위 4비트와 하나의 하위 4비트끼리 곱셈한 결과가 누적합으로 동작한다.
T=3: T=2일 때와 마찬가지로 SE00에서 두 입력값 중 하나의 상위 4비트와 하나의 하위 4비트끼리 곱셈한 결과가 누적합으로 동작한다.
T=4: 두 input의 상위 4비트끼리의 곱셈 결과가 최종누적합으로 더해지게 된다.

최종 누적합을 만듦과 동시에 다음 SE01, SE10에 입력값을 넘겨주게 되어 T=5부터 SE01, SE10이 SE00과 마찬가지의 동작을 4 사이클동안 하게 된다.

이 방식으로 마지막 SE33이 최종 결과값을 만들어 누적합에 적용시키는 것은 T=41일 때이다.

따라서 하나의 SA가 최종 결과값을 만드는 데에는 총 40 cycle이 걸린다.

그러므로 이론적으로 SA 모듈이 40 사이클 동안 처리할 수 있는 연산량은
**32 OPs/cycle × 40 cycles = 1280 OPs**이다.

실제 연산 측면에서는, 각 SE가 누적합 1개를 만들기 위해 총 8개의 연산(4비트 곱셈 4회, 누적합 4회)을 수행하며, 이러한 누적합이 각 SE당 총 4번 이루어진다. 즉, **1개의 SE가 수행하는 실제 연산량은 8 OPs × 4 = 32 OPs**,
따라서 SA 전체의 실제 연산량은 **32 OPs × 16 SE = 512 OPs**가 된다.

최종적으로, Utilization Ratio는 다음과 같이 계산된다:
**Utilization Ratio = 512 OPs / 1280 OPs = 0.4 = 40%**

**5) Discuss how the overall latency and utilization ratio would change if the peak bandwidth of your design were doubled (2x) and halved (0.5x), respectively.**

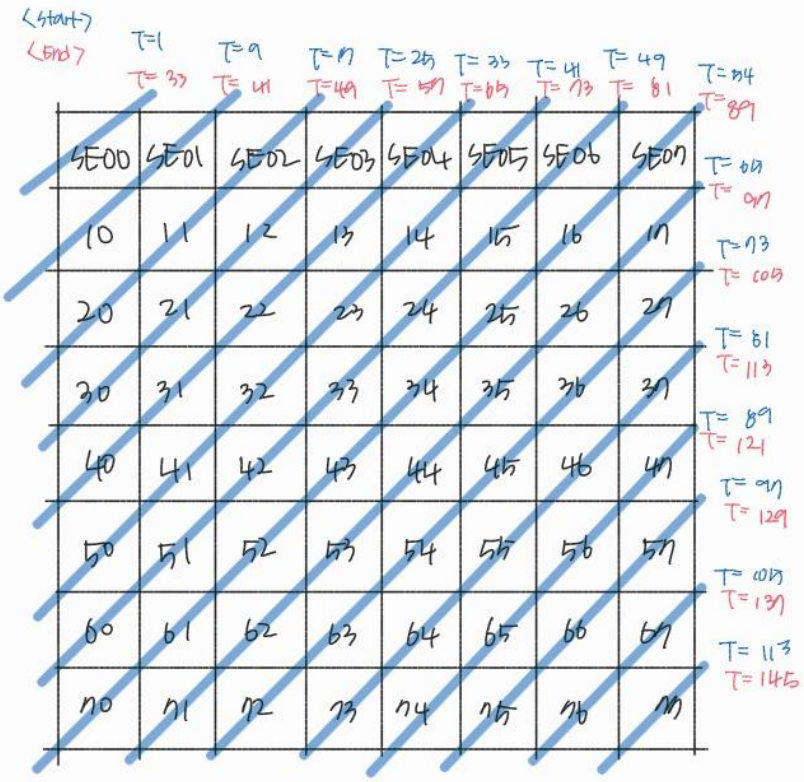
Bandwidth는 메모리로부터 데이터를 읽거나 저장할 때, 단위 시간당 처리할 수 있는 데이터 양을 의미한다. 기존 시스템에서는 한 사이클 동안 하나의 메모리 주소에서 데이터를 읽거나 저장할 수 있도록 구성되어 있었다. 만약 Bandwidth가 2배로 향상된다면, 동일한 시간 내에 두 개의 주소에서 데이터를 동시에 읽고, 두 개의 데이터를 동시에 저장할 수 있게 되므로 LOAD 및 STORE 연산에 소요되는 시간이 절반으로 줄어들게 되어 **Latency가 감소**하게 된다.

반대로, Bandwidth가 절반으로 줄어들 경우, 하나의 데이터에 접근하는 데 2사이클이 걸리게 되어 전체적인 **Latency는 증가**하게 된다.

하지만 Bandwidth의 변화는 메모리 접근 속도에만 영향을 줄 뿐, 내부 연산 구조에는 변동이 없기 때문에, 동일한 4×4 SA 모듈을 사용하는 이상 **Utilization Ratio는 변하지 않고 일정하게 유지될 것**으로 판단된다.

**6) Discuss how the overall latency and utilization ratio would change if the systolic array size of your design were doubled (2x) and halved (0.5x), respectively.**

1. SA (8×8)




(1) latency :

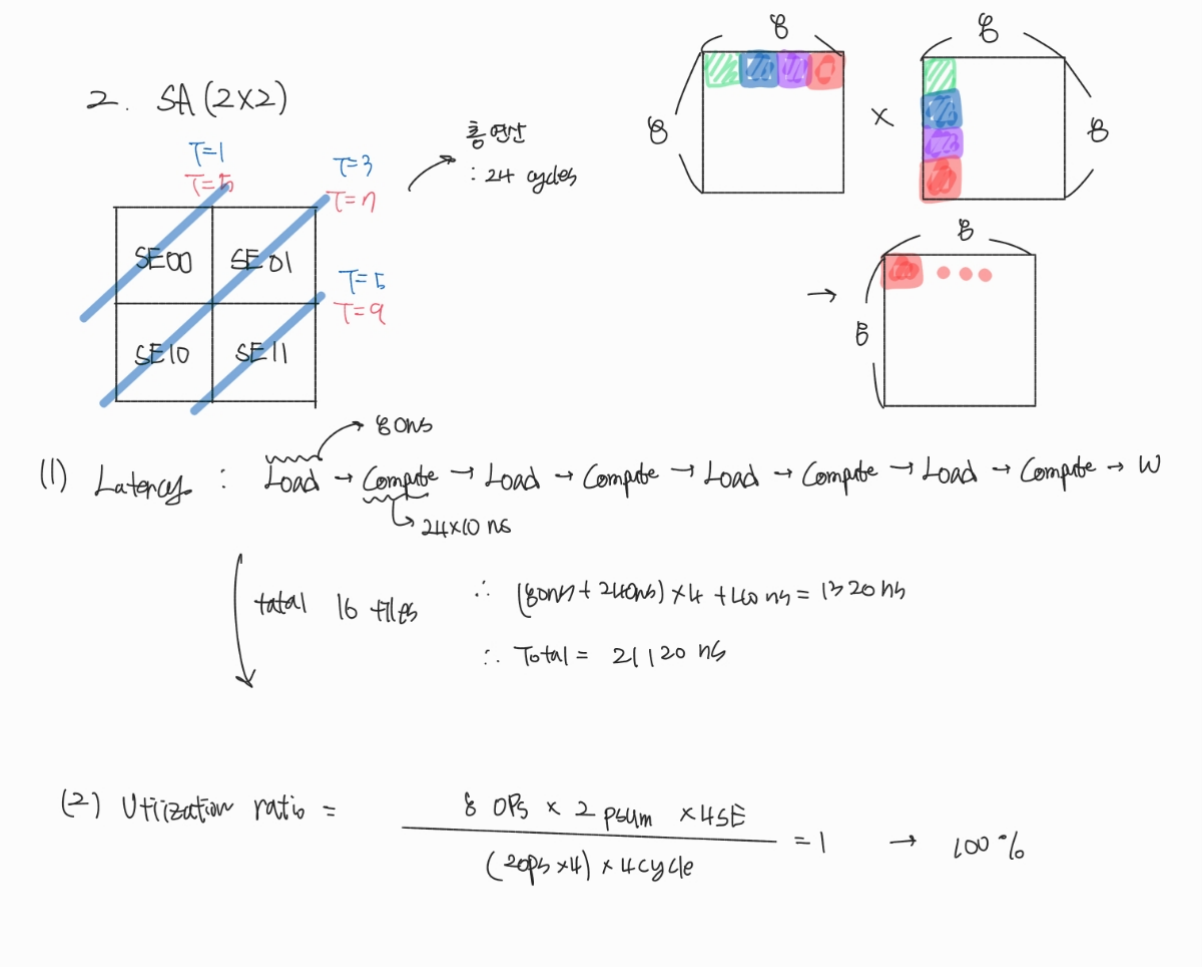
Load → compute → write

$\therefore 64 \times 20ns + 144\ cycles \times 10ns + 64 \times 10ns$

$= 3360\ ns$

(2) Utilization Ratio :

$$\frac{(8 \times 8) \times 4}{(20PS \times 64SE) \times 144\ cycles} = 0.22 \quad \therefore 22.22\%$$




**7) Discuss how you can improve the latency of your design without changing the clock speed.**

**첫째**, 현재 하나의 Systolic Element(SE)에는 MAC 연산기가 1개만 탑재되어 있다고 가정된다. 만약 각 SE에 추가적인 MAC 연산기를 도입한다면, 사이클당 처리할 수 있는 연산량이 증가하게 되어, 특히 COMPUTE 단계에서 발생하는 연산 지연을 줄이는 데 효과적일 수 있다.

**둘째**, 현 시스템에서는 메모리 접근 시 re(read enable) 신호가 we(write enable) 신호에 의존하는 구조로 인해, 동일한 사이클 내에 읽기와 쓰기를 병행할 수 없다. 이를 개선하기 위해 dual-port memory를 도입하면, LOAD와 WRITE를 동시에 처리할 수 있어 전체 Latency를 줄일 수 있는 가능성이 생긴다.

**셋째**, 파이프라인 구조의 최적화를 통해 연산 지연을 줄일 수 있다. 현재 설계에서는 각 타일에 대해 LOAD → COMPUTE → LOAD → COMPUTE → WRITE 단계를

순차적으로 실행하고 있으나, 이를 3단계 파이프라인으로 구성하면 다음 타일의 LOAD와 현재 타일의 COMPUTE를 병렬적으로 수행할 수 있다. 실제 시스템에서도 이러한 병렬 처리가 일부 반영되어 있으며, 이를 보다 체계적으로 구성하면 Latency를 효과적으로 숨길 수 있을 것으로 기대된다.

**8) Discuss how you can improve the utilization ratio of your design.**

현재 시스템의 Utilization Ratio는 약 40% 수준으로, 이는 전체 SE(Systolic Element) 중 상당수가 입력 데이터를 받지 못해 유휴 상태로 남아 있는 시간이 전체 연산 시간의 약 60%에 달한다는 것을 의미한다. 다시 말해, 실제로 연산이 가능한 자원이 충분히 있음에도 불구하고 활용되지 못하는 구간이 많다는 점에서, SE들의 유휴 시간을 줄이는 방향의 개선이 필요하다.

앞서 6번 문제에서도 언급했듯이, SA의 크기가 작아질수록 입력 타이밍에 맞춰 동작하지 못하는 SE의 수가 줄어들어 Utilization Ratio가 높아지는 경향이 관찰되었다. 하지만 SA 사이즈를 줄이면 전체 연산 과정에서 더 많은 Tiling과 LOAD 동작이 필요하게 되고, 한 번에 처리할 수 있는 연산량도 줄어들기 때문에 오히려 전체 Latency가 증가하는 단점이 발생할 수 있다.

따라서, Bandwidth, 입력 데이터의 크기 및 수량 등을 종합적으로 고려하여 적절한 SA 크기를 선정하고, 그에 맞게 Tiling 전략도 조정함으로써 **Utilization Ratio와 Latency 사이의 균형을 맞추고, 전체 시스템의 Peak Performance를 최적화**하는 것이 중요하다.